CS3570: Introduction to Multimedia Technology

Homework #4

Due: 11:59 pm, 2026/05/29

In this homework you will train neural networks that **restore degraded images** corrupted by **heavy JPEG compression plus additive Gaussian noise**. You have learned JPEG compression in this course—block DCT, quantization, zig-zag / RLE / entropy coding. Aggressive quantization gives high compression ratios but introduces visible artifacts: blockiness on 8×8 boundaries, ringing near sharp edges, color bleeding in chroma. Real-world inputs combine these compression artifacts with sensor / channel noise. Your task: given an image that has been JPEG-compressed at `qf` $\in [5, 20]$ and then perturbed with Gaussian noise ($\sigma \in [0, 0.1]$ in $[-1, 1]$ units), train a network to output something as close as possible to the original clean image. Figure 1 shows representative samples drawn from the dataset.



Figure 1: Example of dataset images.

You will implement **two architectures** and run an **ablation study** on each.

The notebook trains in **step units, not epochs** so all runs share an identical compute budget regardless of batch size. The default budget is `MAX_STEPS = 2000` per run. The dataset is a class-balanced subset of ImageNet-1k (50 train + 10 val per class, pre-resized to short-side = 256, stored lossless WebP)—the notebook downloads `train.tar` / `val.tar` automatically from HuggingFace.

All implementation work happens in `HW4.ipynb` (renamed from `HW4_template.ipynb` after you start). Complete every cell marked `# TODO`.

1 Naive CNN with Residual Blocks (35%)

Build a CNN restoration model from a stack of **residual blocks**.

1.1 Examples During Training

Figure 2 shows intermediate restoration results sampled during training—the degraded input, the network's current output, and the clean target, side by side. As training progresses, you can see the model learning to smooth the 8×8 block artifacts and suppress the additive noise, gradually recovering edges and textures from the corrupted input.



Figure 2: Intermediate samples during training: the model learns to smooth the block artifacts and eliminate the noise.

1.2 Design Rules

Mandatory (must hold for any submission).

- a stack of **residual blocks** of the form `y = x + f(x)`;
- spatial dimensions preserved end-to-end (`H_out == H_in`, `W_out == W_in`).

Free to explore. The `# TODO` skeleton in the notebook is one valid implementation, not a contract. You are encouraged to deviate—e.g. swap the activation, add normalization, use bottleneck blocks, mix kernel sizes, try grouped / depthwise / dilated convolutions, tweak initialization, etc.—as long as the mandatory rules still hold.

You will run **five training runs** on this architecture: a **base** configuration, three **single-axis ablations** (size, loss, and one other hyperparameter of your choice), and a **final "best"** configuration that you justify based on what you observed.

Fill in the `p1_results` table at the end of Part 1 (five rows: `base`, `size_*`, `loss_*`, `hp_*`, `best`). The cell prints a comparison and saves `output/p1_results.png`.

Table 1: Part 1 — Naive CNN ablation study (five runs).

Sub-section	Requirement	Points
1(a) Implementation + Base run	Implement `ResBlock` and `NaiveCNN` in the cells marked `# TODO`. Train **one base configuration** with `MAX_STEPS = 2000` and the default loss (`mse + 0.1 * ssim`). Report the final `val_psnr` and save `output/p1-naivecnn-base-sample.png`.	**8%**
1(b) Size ablation	Re-train with **at least one alternative size** by changing one or more of `HIDDEN_DIM`, `NUM_BLOCKS`, `KERNEL_SIZE`. Keep loss / optimizer / schedule identical to the base run. Save `output/p1-naivecnn-size_<tag>-sample.png`.	**8%**
1(c) Loss ablation	Re-train with **at least one alternative** `LOSS_CFG` (see SenseCraft). Keep size and other hyperparameters identical to the base run.	**8%**
1(d) Hyperparameter ablation	Pick **one** other hyperparameter (`LR` / `WEIGHT_DECAY` / `WARMUP_STEPS` / `TRAIN_BATCH_SIZE` / `MAX_STEPS` / ...) and re-train with **at least one alternative value**, keeping the rest identical to the base run.	**8%**
1(e) Best configuration	Combine the lessons from (b)/(c)/(d) into the configuration **you believe will perform best** at the same compute budget. Train it, and justify the choice in 2–3 sentences.	**3%**

2 UNet (35%)

A multi-scale architecture with skip connections. The notebook walks through a clean **recursive** implementation:

```
                      skip
input_blocks ---------------------> merge -> output_blocks
            \                      /
             down -> mid -> up
```

where `mid` is either another `UNetStage` (with `depth - 1`) or `nn.Identity()` at the bottom, and the down / up paths use `nn.PixelUnshuffle` / `nn.PixelShuffle`.

2.1 Design Rules

Mandatory (must hold for any submission).

- **multi-resolution processing**—features must pass through at least one *downsample* → *bottleneck* → *upsample* path; the number of scales is up to you;
- **skip connections** between matching encoder / decoder resolutions;
- spatial dimensions preserved end-to-end.

Free to explore. The recursive `UNetStage` skeleton is one clean way to do it, but you may:

- implement the multi-scale loop iteratively or unrolled instead of recursively;

- use strided conv or pooling for the down path (instead of `PixelUnshuffle`);
- use transpose conv or upsample + conv for the up path (instead of `PixelShuffle`);
- add gating / attention / extra residual connections inside stages;
- vary the channel count across scales (classic UNet doubles channels each level);

. . . as long as the mandatory rules still hold.

You will run the **same five-run ablation study** as Part 1.

Table 2: Part 2 — UNet ablation study (five runs).

Sub-section	Requirement	Points
2(a) Implementation + Base run	Implement `UNetStage` and `UNetRestoration`. Train one base configuration.	**8%**
2(b) Size ablation	Vary `HIDDEN_DIM` / `DEPTH` / `BLOCKS_PER_STAGE`.	**8%**
2(c) Loss ablation	Vary `LOSS_CFG`.	**8%**
2(d) Hyperparameter ablation	Pick one other hyperparameter (it may differ from the one chosen in 1(d)).	**8%**
2(e) Best configuration	Pick and justify your best config, then train it.	**3%**

Fill in the `p2_results` table at the end of Part 2.

After training the **best Part 1** and **best Part 2** models, run the comparison cell—it produces `output/p1_vs_p2.png`, which is used in the report.

3 Report (30%)

Write the report inside the notebook using markdown. Every section must combine **quantitative numbers (PSNR + SSIM)** with a **qualitative discussion** that references the saved sample images—describing artifacts, textures, edges, smoothness, and so on. Numbers without a visual discussion (or vice versa) receive only partial credit.

I. **Part 1 ablations + best (9%).** For each axis (size / loss / hp), report the direction and magnitude of the change in PSNR **and** SSIM, plus what changed visibly in the samples. Justify your `1(e)` best configuration.

II. **Part 2 ablations + best (9%).** Same structure for the UNet runs.

III. **Part 1 vs. Part 2 (9%).** Using your two best runs, compare validation PSNR, validation SSIM, parameter count, and **what the multi-scale UNet visibly fixes that the flat residual stack misses (or does not)**—point to specific spots in the `output/p1_vs_p2.png` figure. Cases where the metrics and the visuals disagree are themselves worth discussing.

IV. Reflection (3%). Of the two architectures, which would you actually deploy for compression-artifact + noise restoration on phone photos, and why? Consider PSNR, SSIM, parameter count, inference cost, *and* the qualitative behavior you observed.

Reminder

- You are not allowed to use any function that directly generates the result for each step, except for those provided by us (PyTorch, Lightning, SenseCraft, and AnySchedule are all allowed; do not use a pre-trained restoration model).
- Your code must display and output your results so that we can verify its correctness. The notebook saves rolling preview images and per-run final samples to `output/` automatically—keep them for the submission.
- Please follow the instructions in the Jupyter notebook and complete the parts marked as "TODO".
- Please include your report in the Jupyter notebook using markdown.
- **Please compress your code and result images into a zip file named `HW4_{StudentID}.zip` and upload it to eeclass.** Do *not* include the dataset or trained checkpoints (they are too large and are reproducible from your code).
- Homework should be submitted before the announced due time. Scores of late submissions will be reduced by 20% per day.
- If you encounter any problems or have questions, please post them on eeclass.
- Please follow the file structure below.

```
HW4_{StudentID}
├── HW4.ipynb
└── output
    ├── dataset_preview.png
    ├── p1-naivecnn-base-sample.png
    ├── p1-naivecnn-size_<tag>-sample.png
    ├── p1-naivecnn-loss_<tag>-sample.png
    ├── p1-naivecnn-hp_<tag>-sample.png
    ├── p1-naivecnn-best-sample.png
    ├── p1_results.png
    ├── p2-unet-base-sample.png
    ├── p2-unet-size_<tag>-sample.png
    ├── p2-unet-loss_<tag>-sample.png
    ├── p2-unet-hp_<tag>-sample.png
    ├── p2-unet-best-sample.png
    ├── p2_results.png
    └── p1_vs_p2.png
```